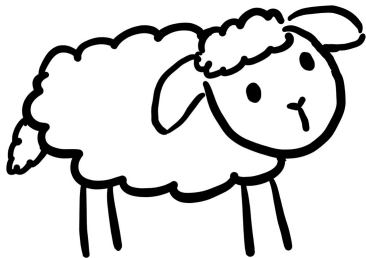


Greedy Algorithms

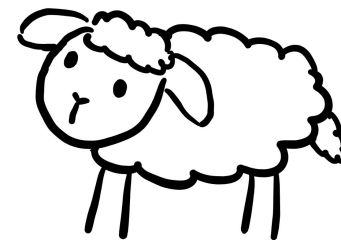


Progress on APSP since Floyd-Warshall

Author	Runtime	Year
Fredman	$n^3 \log \log^{1/3} n / \log^{1/3} n$	1976
Takaoka	$n^3 \log \log^{1/2} n / \log^{1/2} n$	1992
Dobosiewicz	$n^3 / \log^{1/2} n$	1992
Han	$n^3 \log \log^{5/7} n / \log^{5/7} n$	2004
Takaoka	$n^3 \log \log^2 n / \log n$	2004
Zwick	$n^3 \log \log^{1/2} n / \log n$	2004
Chan	$n^3 / \log n$	2005
Han	$n^3 \log \log^{5/4} n / \log^{5/4} n$	2006
Chan	$n^3 \log \log^3 n / \log^2 n$	2007
Han, Takaoka	$n^3 \log \log n / \log^2 n$	2012
Williams	$n^3 / \exp(\sqrt{\log n})$	2014



This is wild!



Conclusion: Maybe $O(n^{2.99})$ is impossible?

Maybe $O(n^{2.99})$ is impossible?

Either **ALL** of the following have $O(n^{<3})$ time algorithms or **NONE** of them do: (Virginia Vassilevska Williams, Ryan Williams, 2010)

1. APSP
2. Minimum Weight Triangle
3. Metricity
4. Minimum Cycle
5. Distance Product
6. Second Shortest Path
7. Replacement Paths
8. Negative Triangle Listing
- ...



Greedy Algorithms

Pick the best choice **NOW**.

Prove you end up with an optimal solution.

Proof Technique: “Exchange” argument



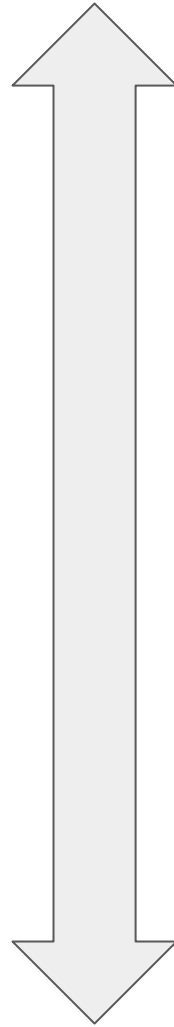
Warning: Greed is generally bad!



Greed

Divide and conquer

Dynamic programming



- Fast
- Doesn't work for most problems
- Often slower than greed and faster than DP
- Works when solutions to disjoint subproblems can be combined into final solution
- Generally slower (but still usually efficient)
- Applies to many problems

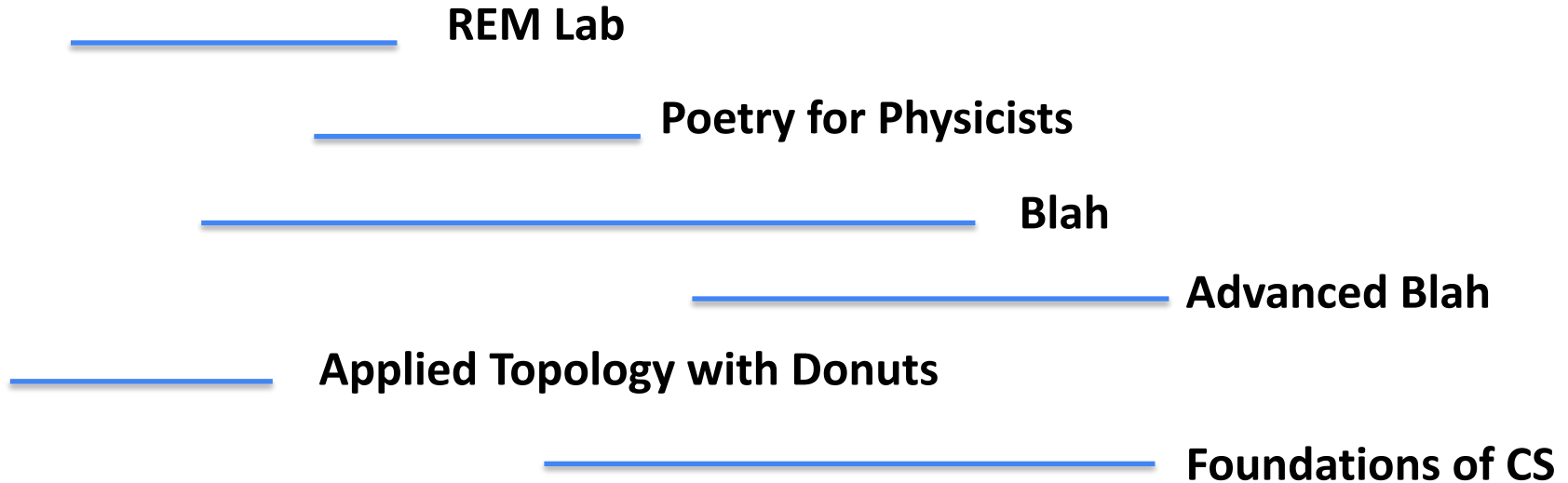
⋮

harder problems where none of these methods apply (coming soon)

But sometimes greed can be good...

Unweighted Course Registration

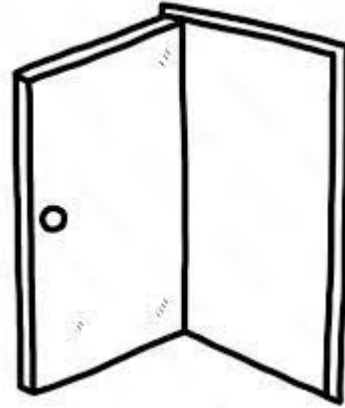
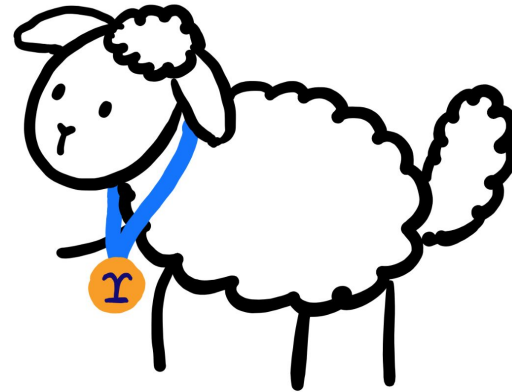
(aka Task Scheduling)



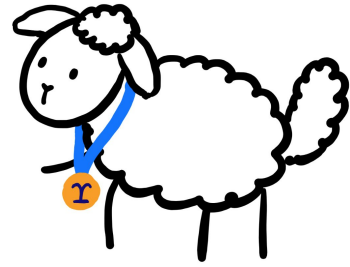
Goal: Choose a **largest*** number of non-intersecting courses
(there may be many optimal solutions, we just seek one!)

*We do not endorse this course registration strategy

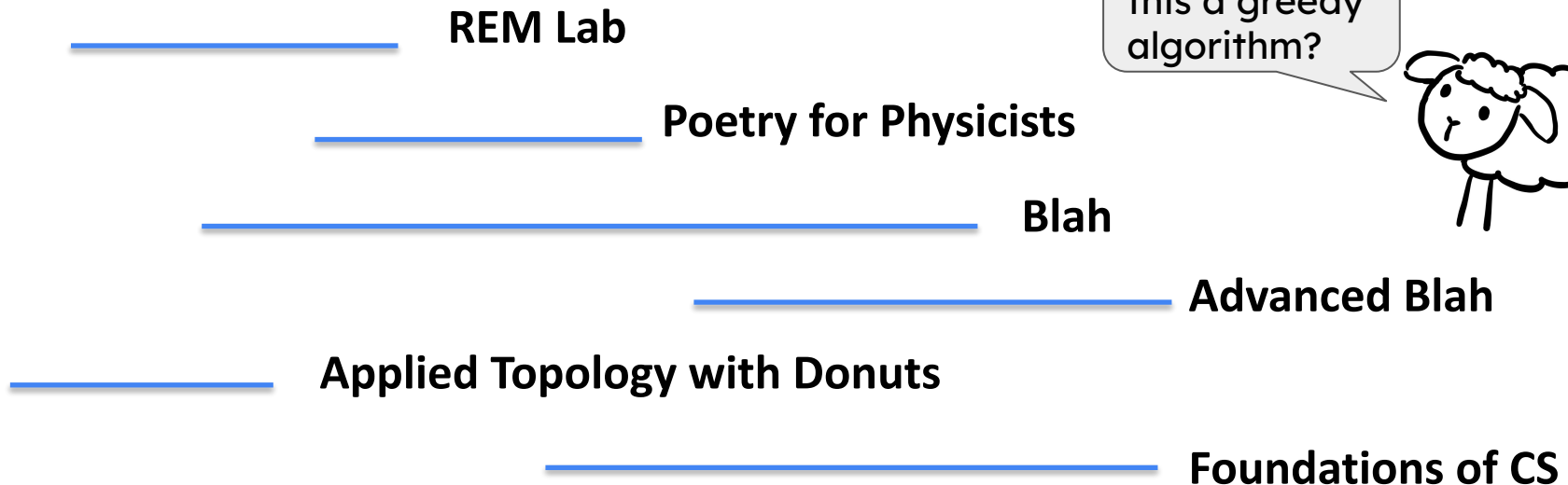
I have not 1, not 2, but 3
greedy algorithms!



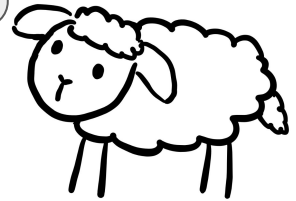
Professor Υ 's Greedy Algorithms



Attempt 1: Choose a **shortest interval**, take it, remove overlaps, iterate on remaining problem.

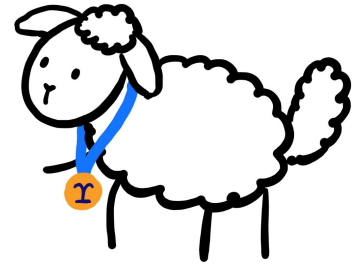


What makes
this a greedy
algorithm?



Counterexample:

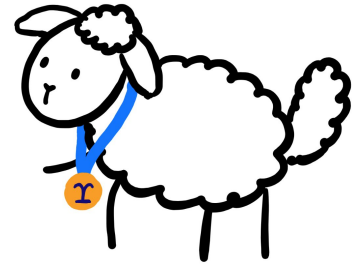
Professor Υ 's Greedy Algorithms



Attempt 2: Choose an **interval that starts earliest**, take it, remove overlaps, iterate on remaining problem.

Counterexample:

Professor Υ 's Greedy Algorithms



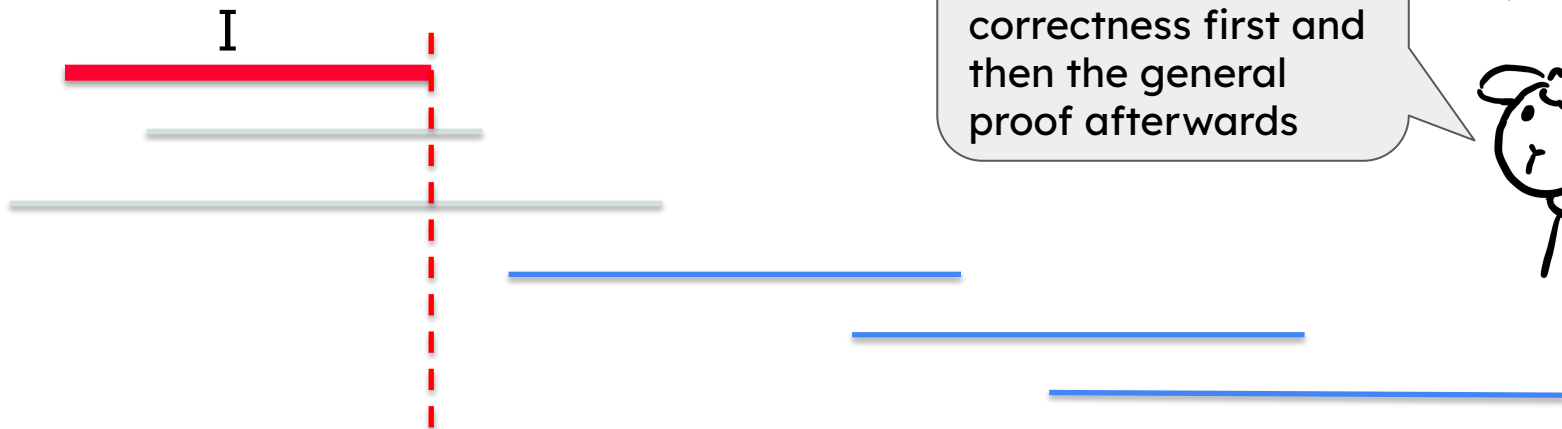
Attempt 3: Choose an **interval that overlaps with the fewest other intervals**, take it, remove overlaps, iterate on remaining problem.

Counterexample:

A Correct Greedy Algorithm: “Earliest Ending Time (EET)”

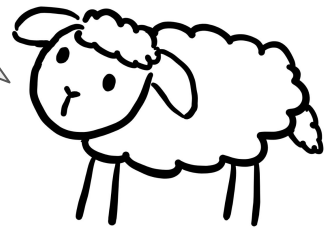
- Sort the intervals by *ending time*
- Greedily take an **interval I that ends first** (break ties arbitrarily)
- Remove the intervals that overlap with the one just selected
- Iterate on remaining problem

Running time:



Let's see the key
idea of the proof of
correctness first and
then the general
proof afterwards

This is the whole
algorithm.
Everything after
this is analysis.



Proof that first chosen interval is “safe”

Key Claim: An interval **I** that **ends first** is a **safe** choice, i.e., it is in some optimal solution.

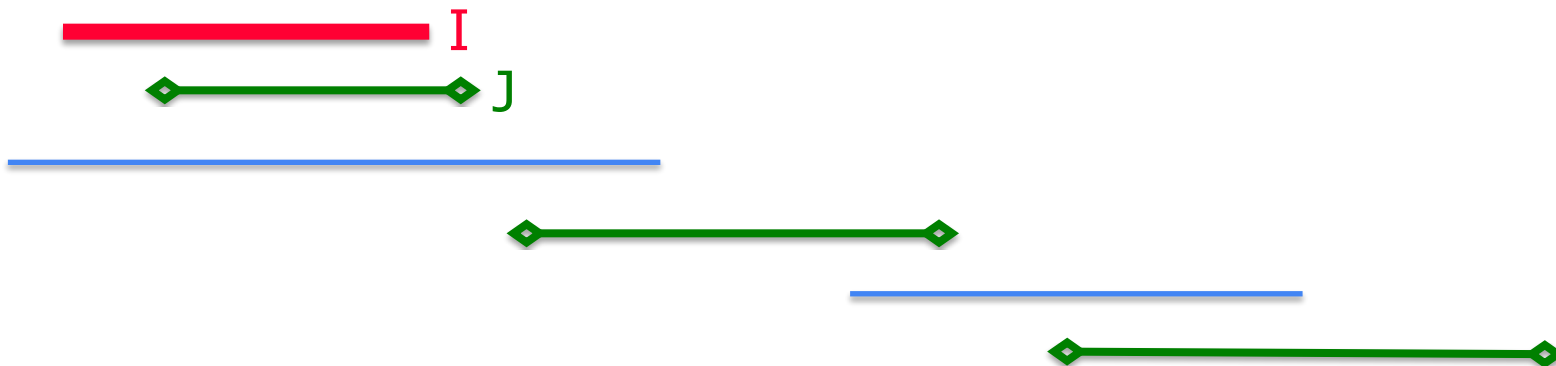
Proof of Claim: Let **J** be an interval that **ends first** in **OPT**.
If **I=J**, we're done.

an optimal solution

Otherwise, take **OPT** and **exchange J** out for **I** to form **OPT'**.

Want to show: **OPT'** is an optimal solution containing interval **I**!

- Why is **OPT'** of optimal size?
- Why is **OPT'** a valid solution (no overlaps)?



Proof that **all** chosen intervals are “safe”

Let **ALG** be the output of the EET algorithm. If **ALG**=**OPT**, we're done.

If **ALG**≠**OPT**, consider the **first** interval I_{diff} of **ALG** that's **not** in **OPT**.

$I_1, I_2, \dots, I_{\text{diff}} \dots = \text{ALG}$ (ordered by end time)

$\underbrace{I_1, I_2, \dots}_{\text{same as ALG}}, \underbrace{J_1, J_2, \dots}_{\text{other intervals (may or may not be in ALG, no particular order)}} = \text{OPT}$

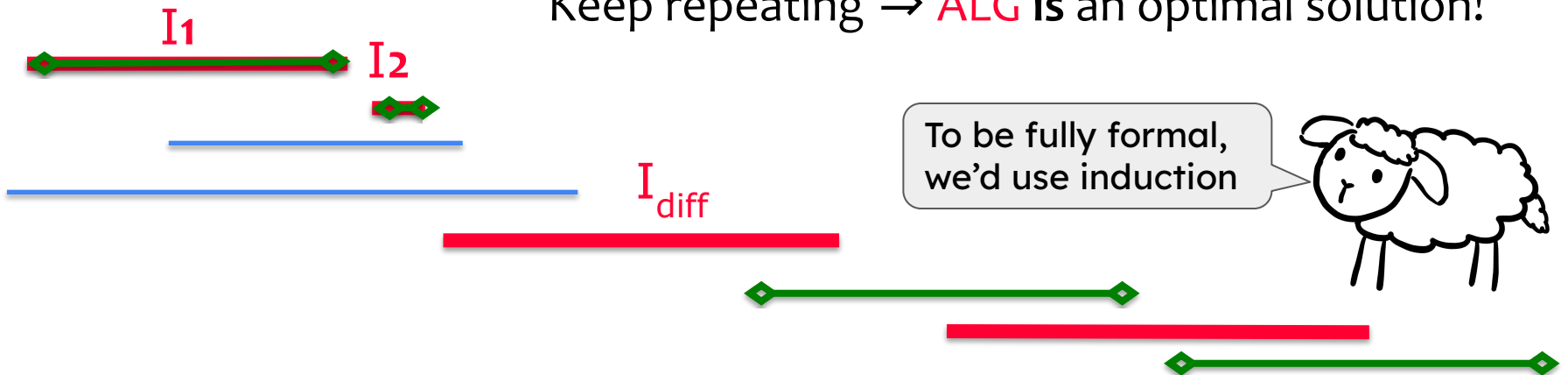
(will prove)

Key Claim: There's an optimal solution **OPT'** containing $I_1, I_2, \dots, I_{\text{diff}}$.

Consequence of Claim: We moved the “first difference” point later!

(I.e. first difference point of **ALG**→**OPT'** is later than that of **ALG**→**OPT**)

Keep repeating $\Rightarrow$ **ALG** is an optimal solution!



Proof that **all** chosen intervals are “safe”

Key Claim: There's an optimal solution OPT' containing $I_1, I_2, I_3, \dots, I_{\text{diff}}$.

Proof of Claim: Let J be the **first-ending** interval in OPT that's **not** in ALG .

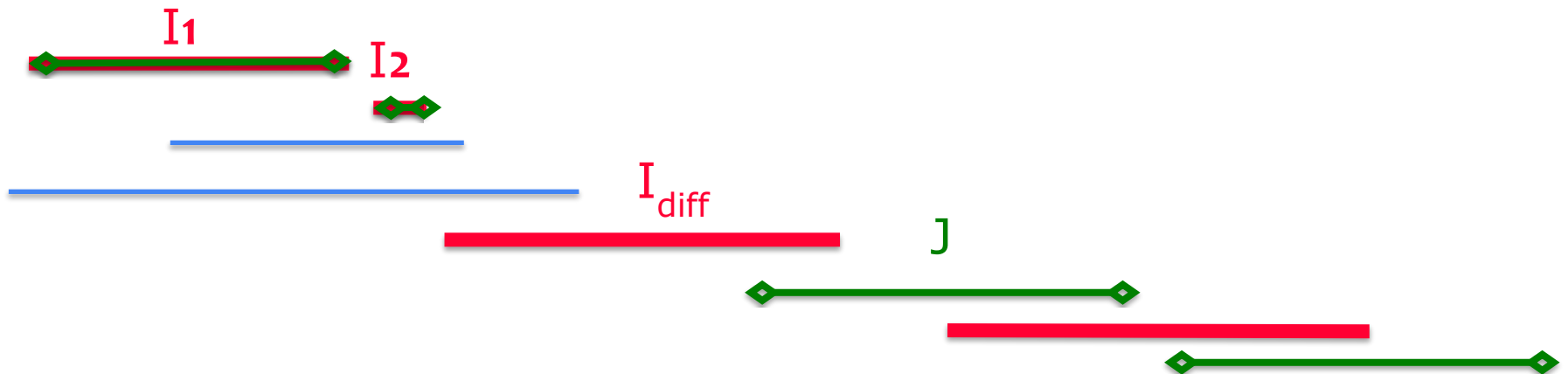
Take OPT and **exchange** J out for I_{diff} to form OPT' .

→ Why is OPT' of optimal size?

→ Why is OPT' a valid solution? Why doesn't I_{diff} overlap with...

along with I_{diff} these cover OPT' {

- OPT intervals before J ?
- OPT intervals after J ?



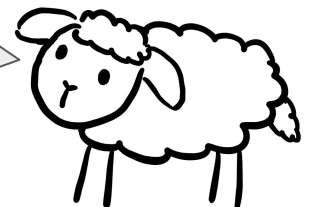
General strategy commonly used for analyzing greedy algorithms:

Exchange argument

The idea: Show that we can transform any **optimal solution** into the **solution given by our algorithm** by **exchanging** each piece of it out one-by-one so that:

- the value doesn't become worse
- the solution remains valid

All this exchanging is the analysis of correctness, not part of the algorithm itself!

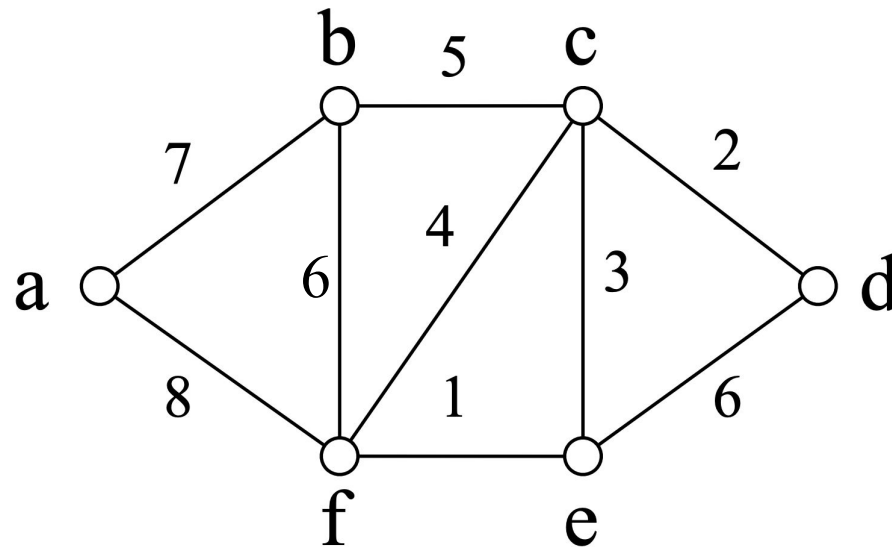


Another problem where greed is good...

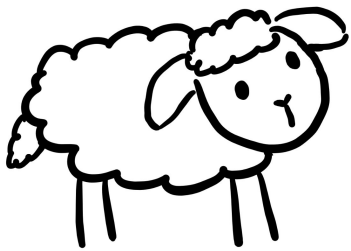
A Highway Problem

Input: an undirected connected graph with positive edge weights
e.g. a set of cities and distances between them

Output: minimum total length of highway to connect all cities
i.e. it should be possible to reach any city from any other using
just the highways



a and **d** are
Ann Arbor and
Detroit



Another problem where greed is good...

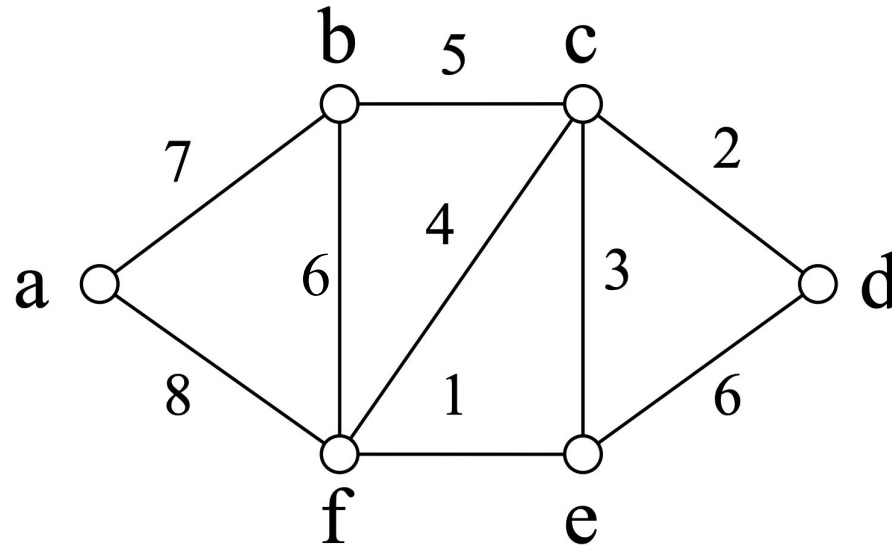
A Highway Problem

Claim: Solution is a tree.

↙ This fact will be useful later too

Why? If a connected graph has a cycle, we can delete an arbitrary edge from the cycle and still have a connected graph.

Definition (review): A tree is a connected graph with no cycles.

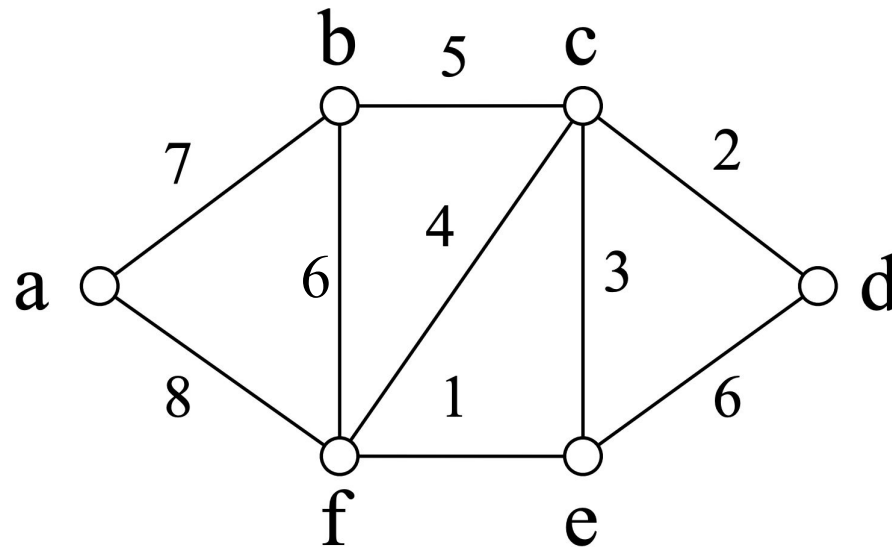


The Highway Problem is commonly known as:

Minimum Spanning Tree (MST)

Given a graph G , a **spanning tree** is a subgraph of G that is spanning (includes all vertices) and is a tree.

MST problem: Given an undirected connected graph with positive edge weights, find a minimum-weight spanning tree.

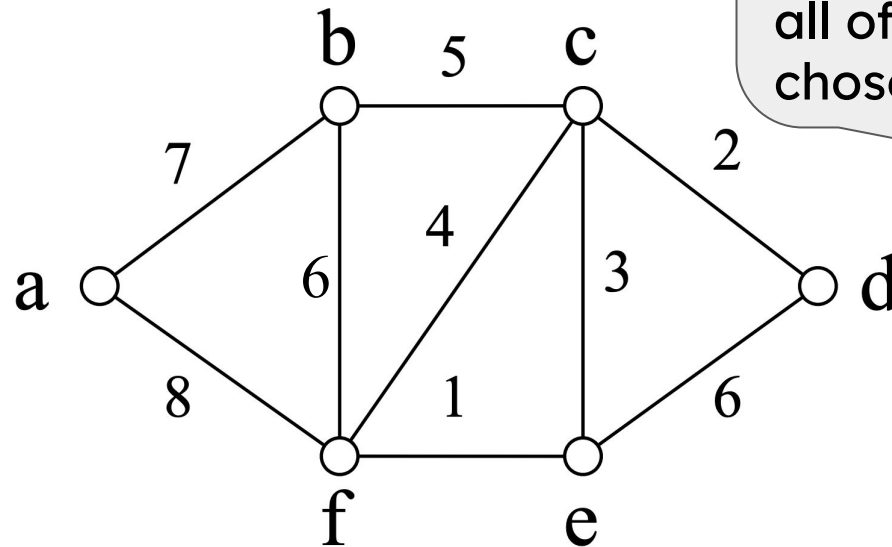


Minimum Spanning Tree (MST)

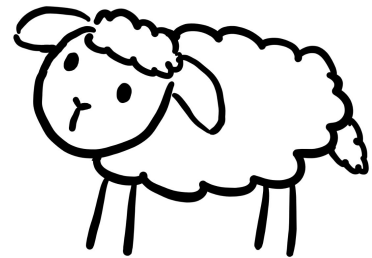
Template for greedy algorithm:

Greedily choose an edge and add it to our MST. Repeat.

But which edge do we choose?



We need to choose a **safe** edge: an edge that appears in *some* optimal solution with all of the previously chosen edges.



Kruskal's Algorithm (1956)

Choose a minimum weight edge!
(that doesn't introduce a cycle)



Kruskal(G): // G is a weighted, undirected graph

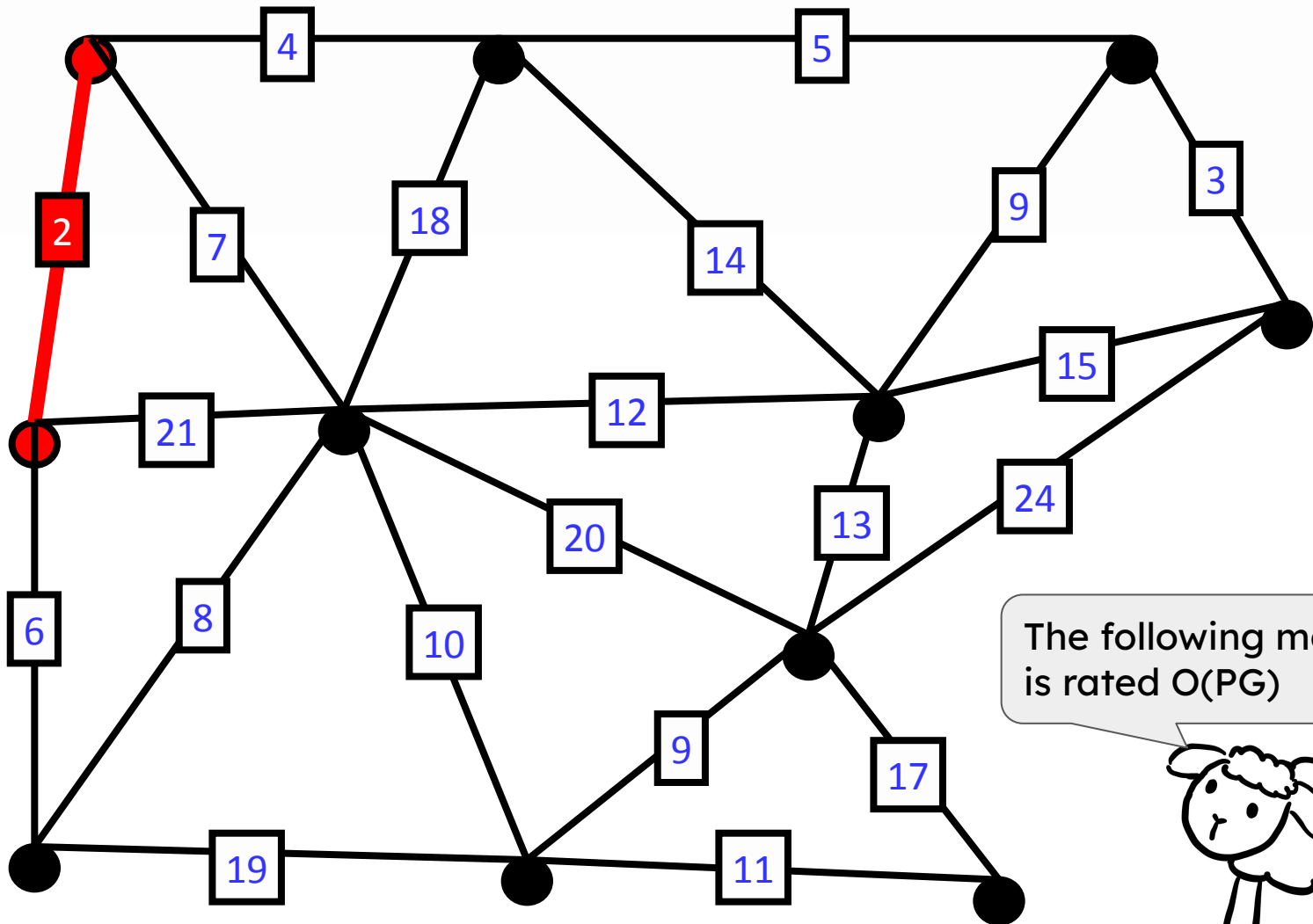
$T \leftarrow \emptyset$ // invariant: T has no cycles

for each edge e in *increasing order of weight*:

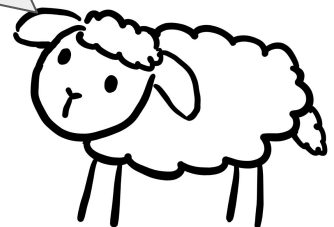
if $T + e$ is acyclic: $T \leftarrow T + e$

return T

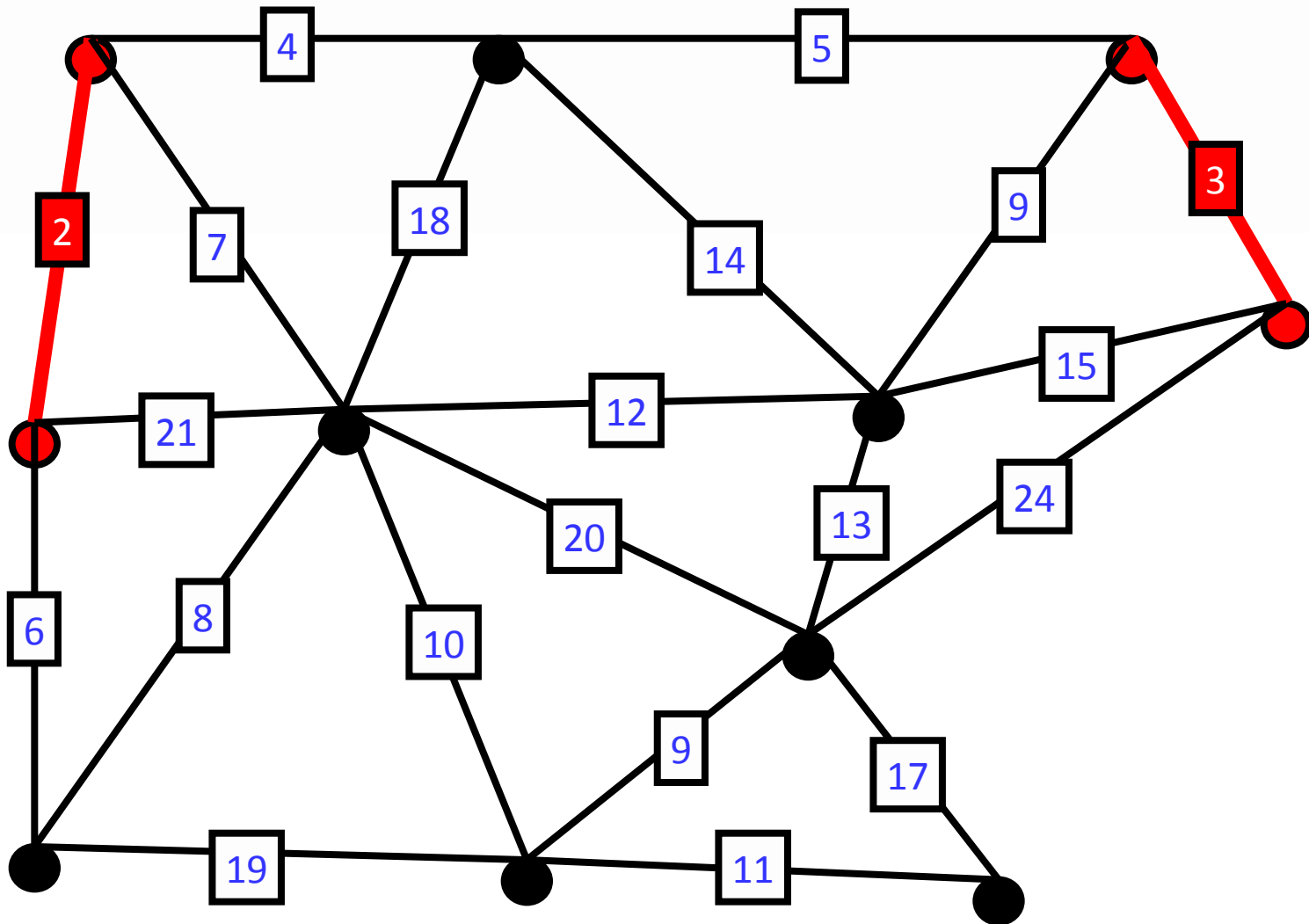
Sorted weights: **2**,3,4,5,6,7,8,9,9,10,11,12,13,14,15,17,18,19,20,21,24



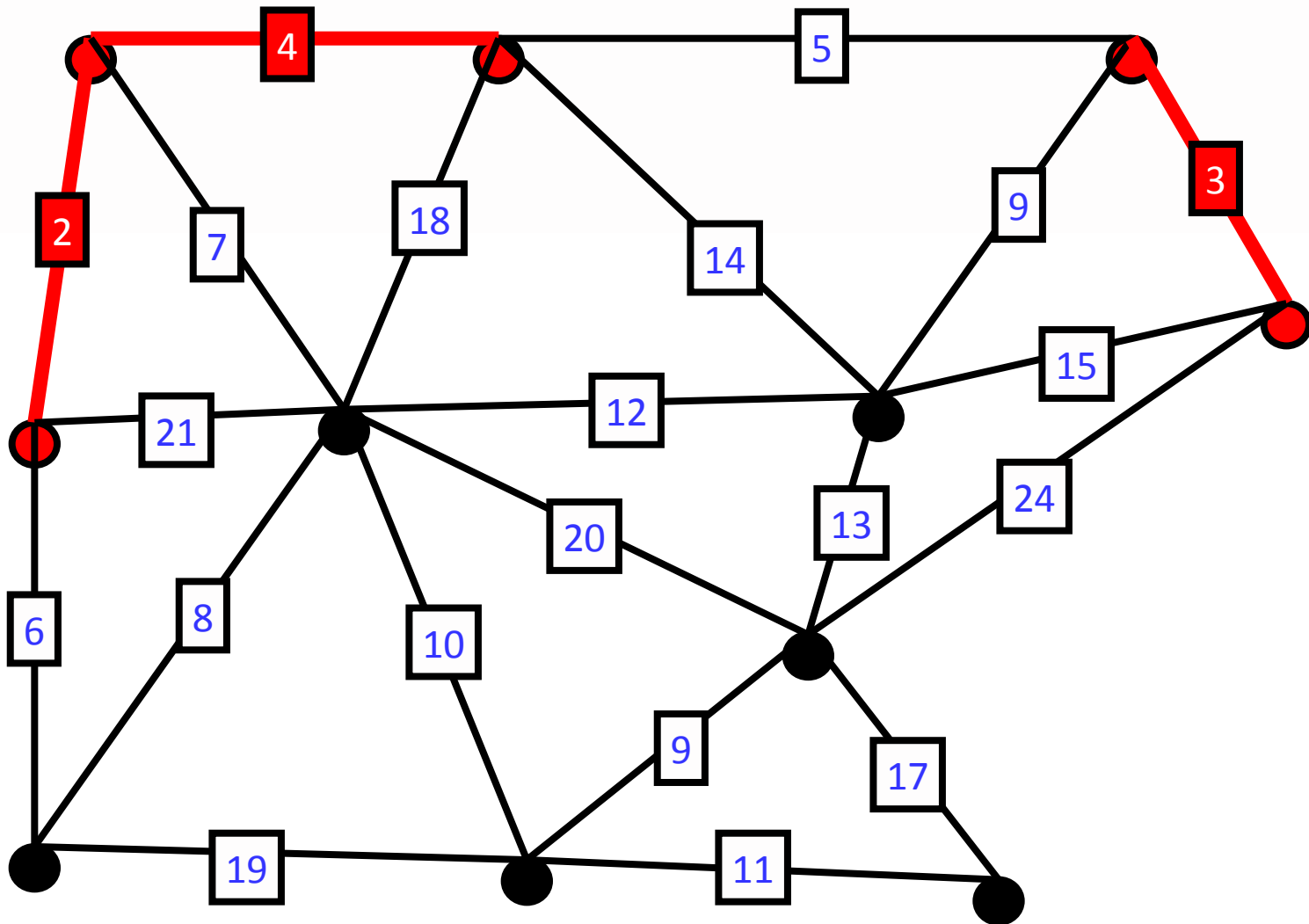
The following movie
is rated O(PG)



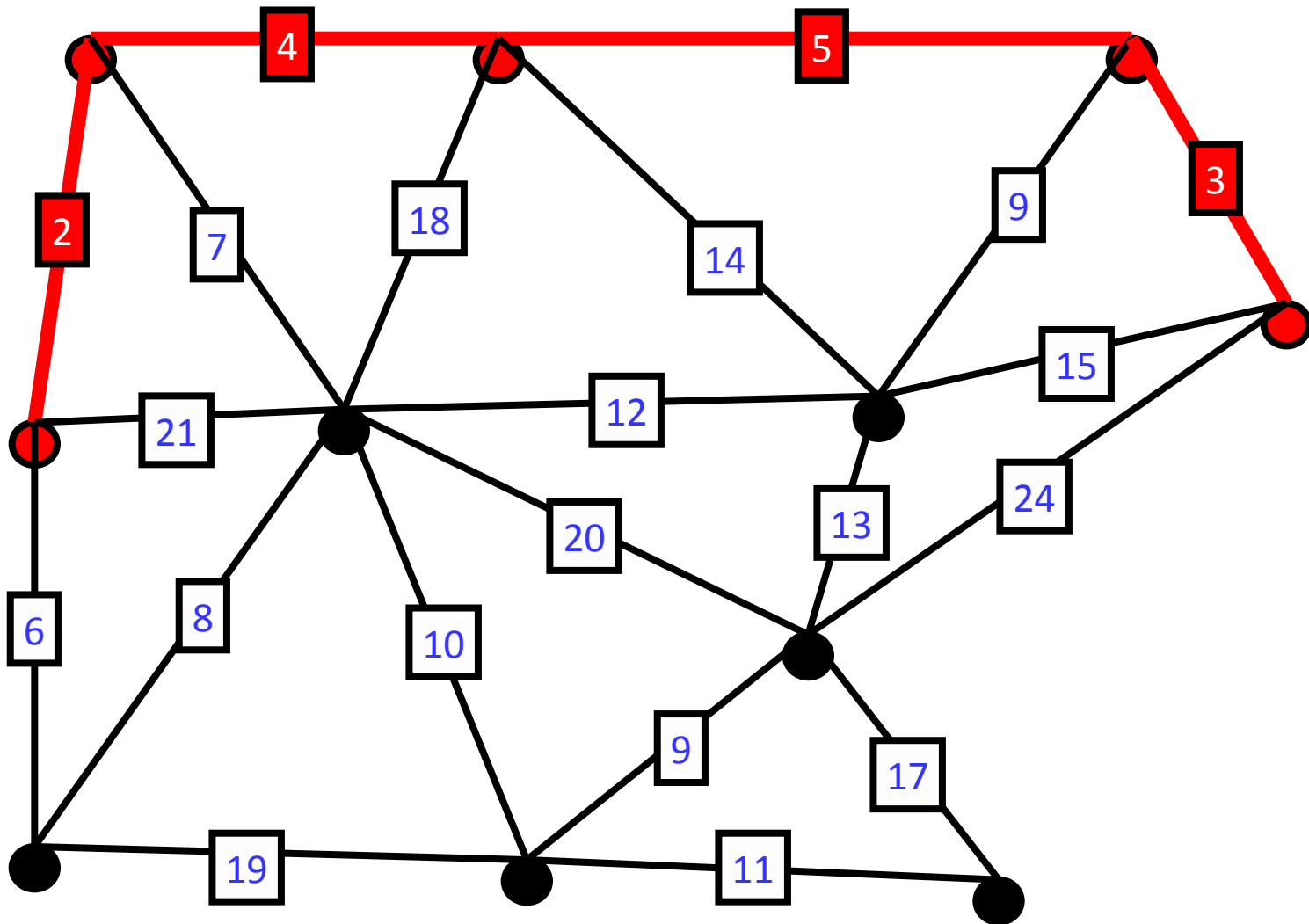
Sorted weights: 2, **3**, 4, 5, 6, 7, 8, 9, 9, 10, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21, 24



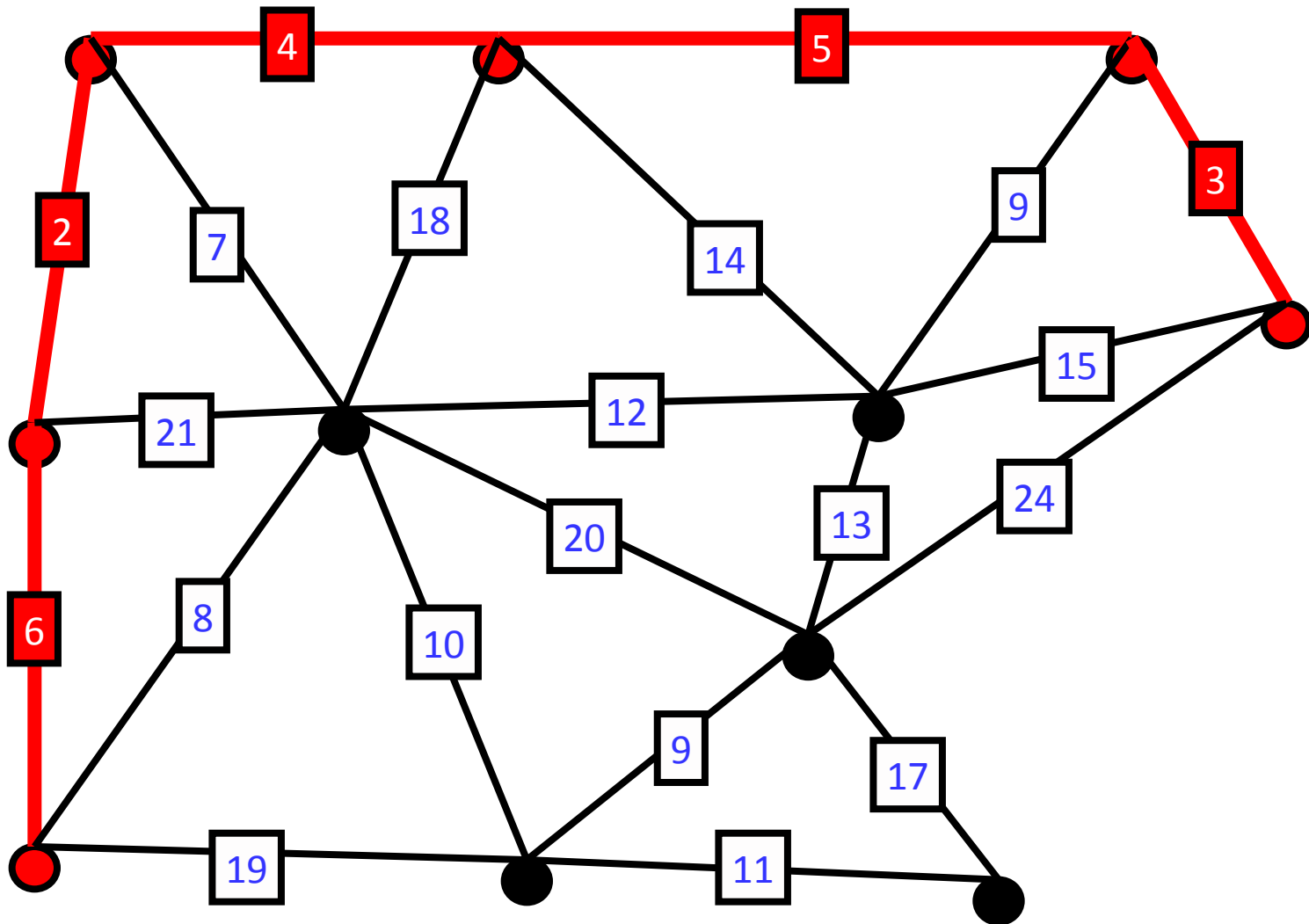
Sorted weights: 2,3,4,5,6,7,8,9,9,10,11,12,13,14,15,17,18,19,20,21,24



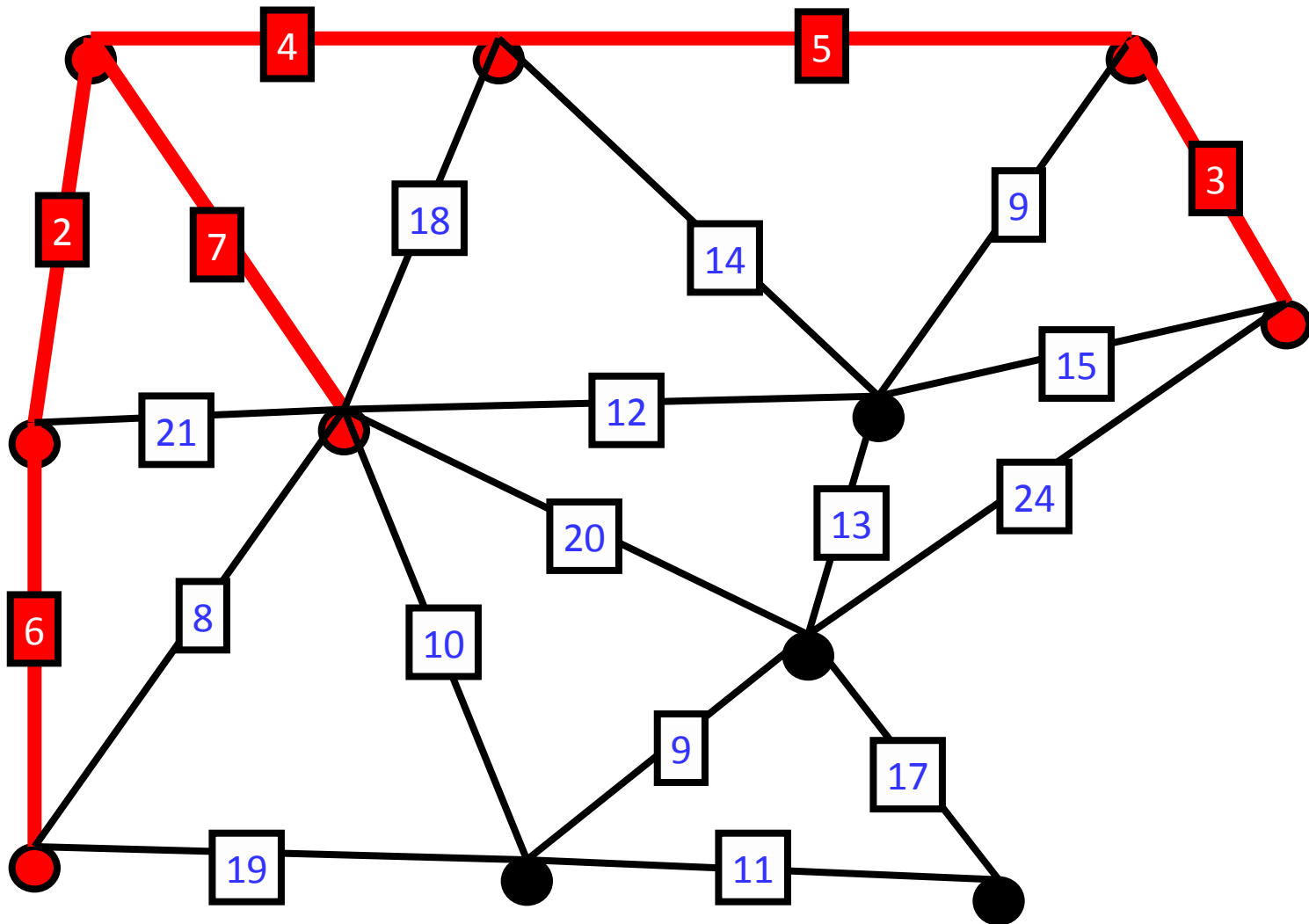
Sorted weights: 2,3,4,5,6,7,8,9,9,10,11,12,13,14,15,17,18,19,20,21,24



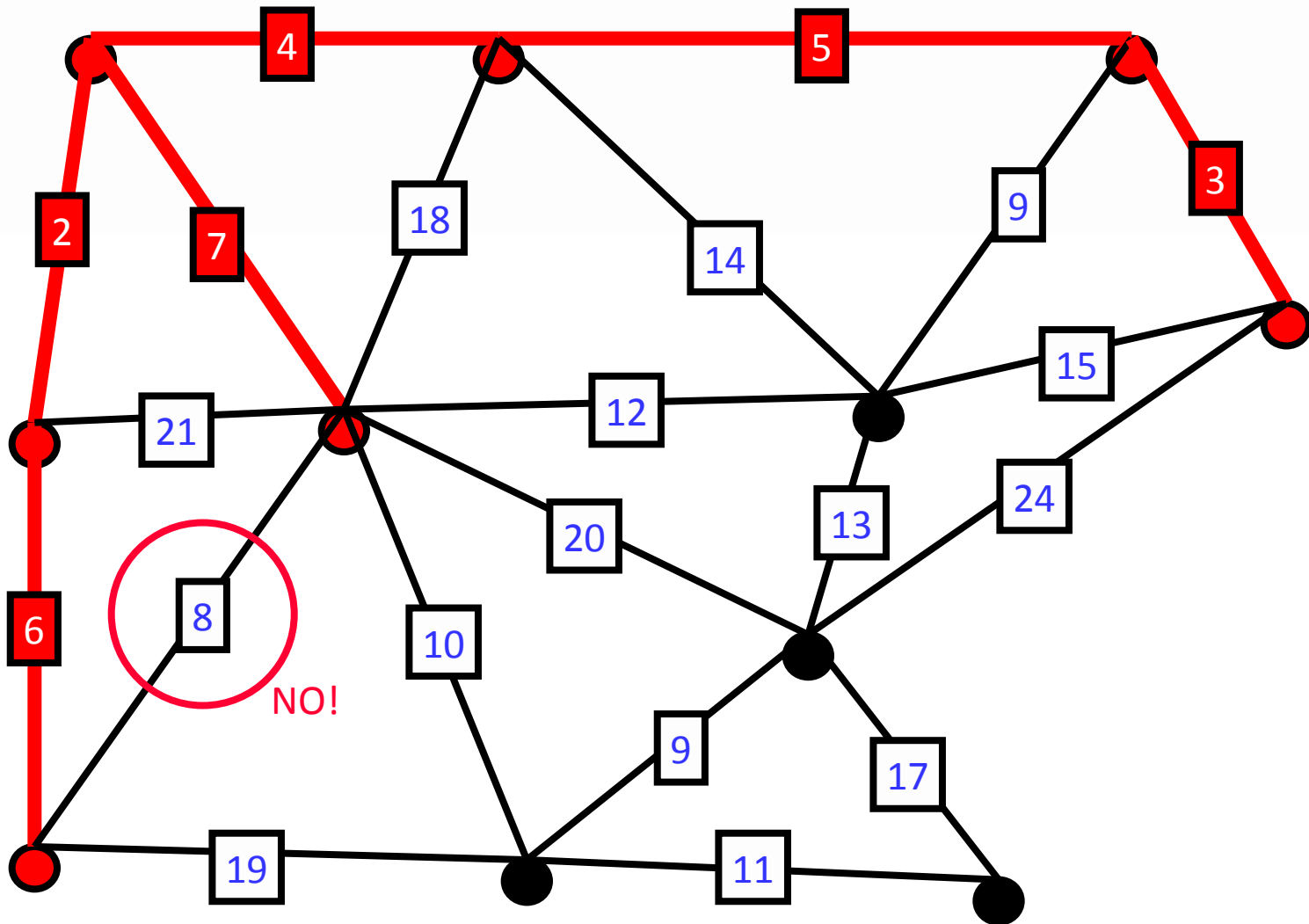
Sorted weights: 2,3,4,5,6,7,8,9,9,10,11,12,13,14,15,17,18,19,20,21,24



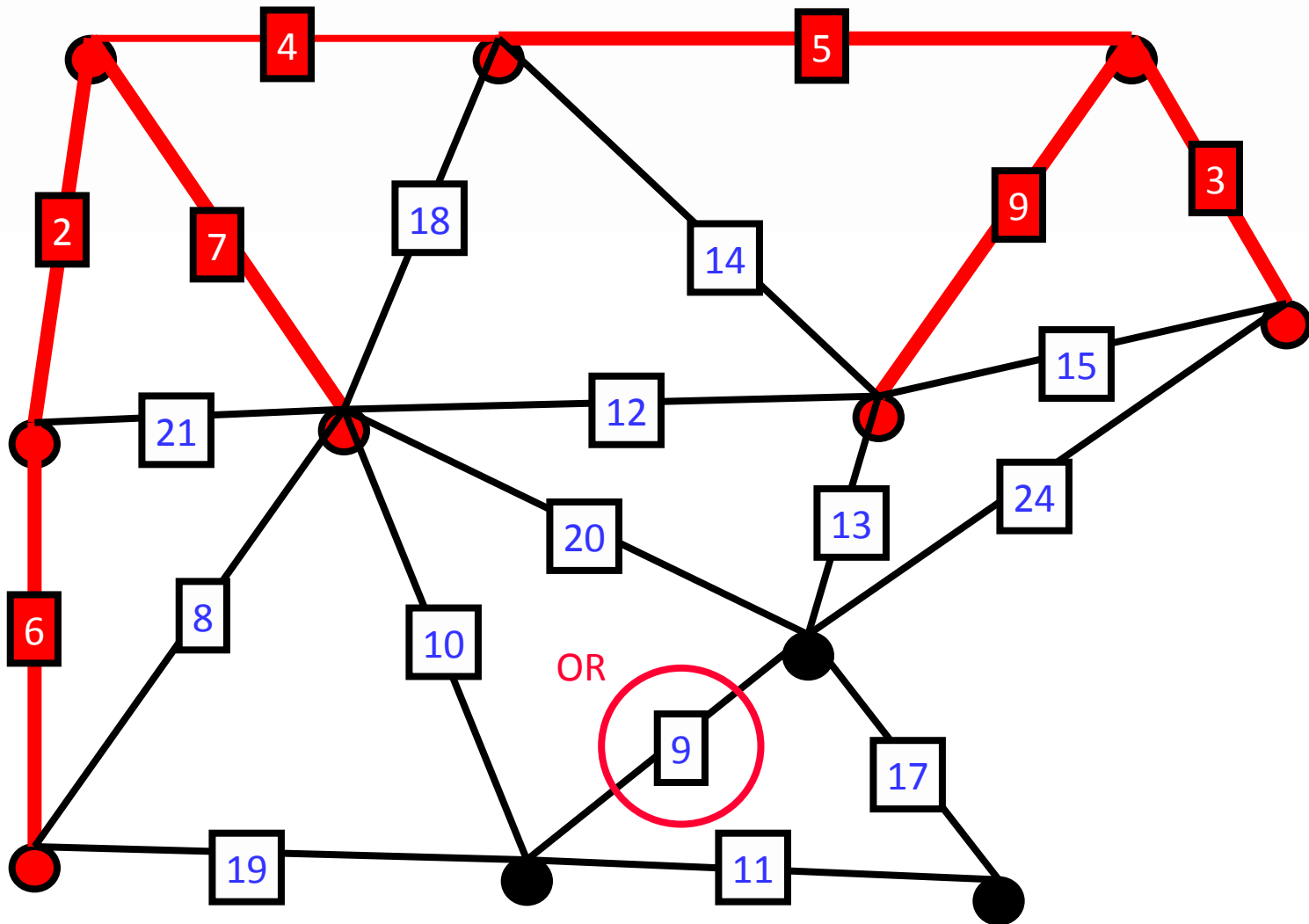
Sorted weights: 2,3,4,5,6,**7**,8,9,9,10,11,12,13,14,15,17,18,19,20,21,24



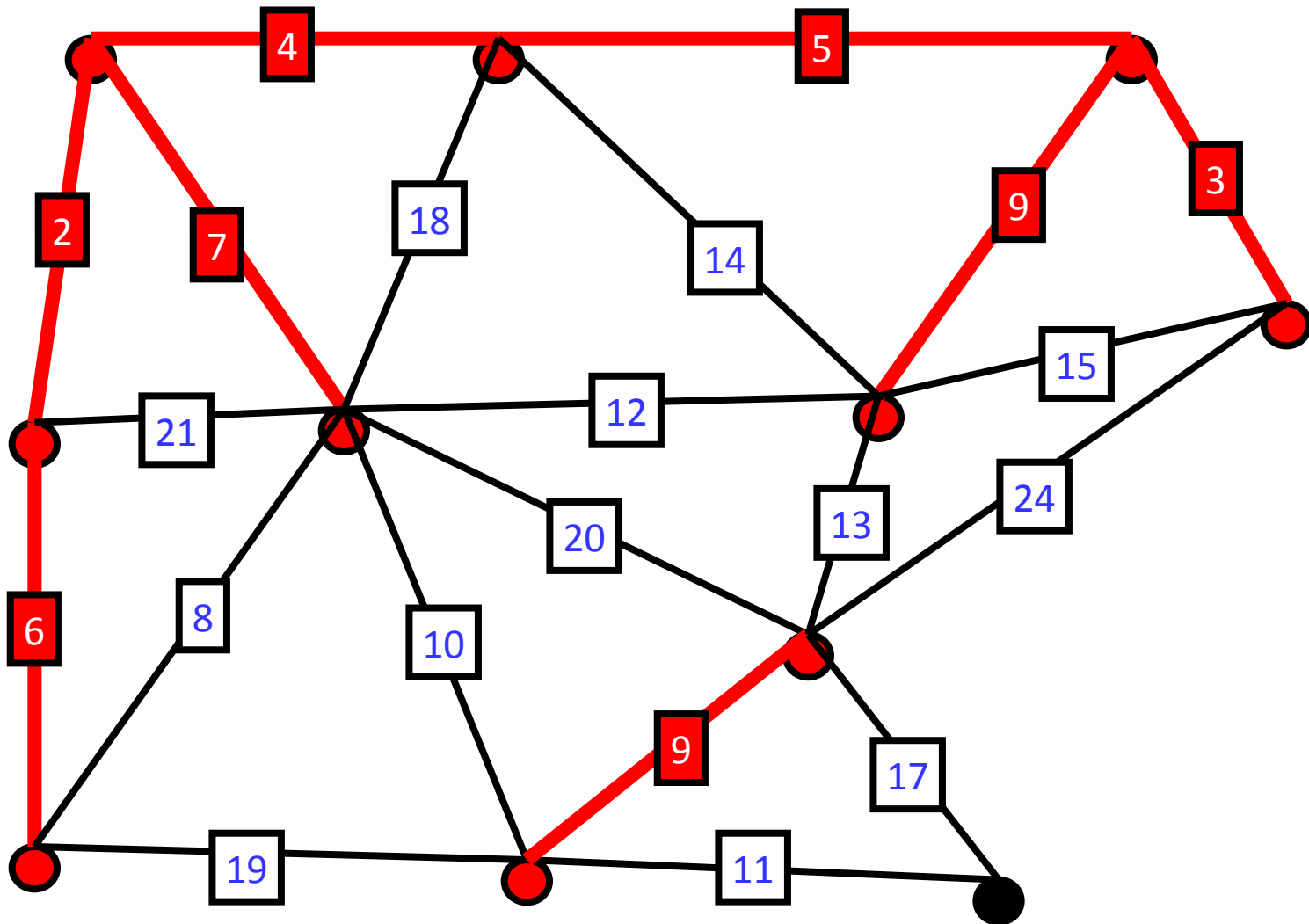
Sorted weights: 2,3,4,5,6,7,8,9,9,10,11,12,13,14,15,17,18,19,20,21,24



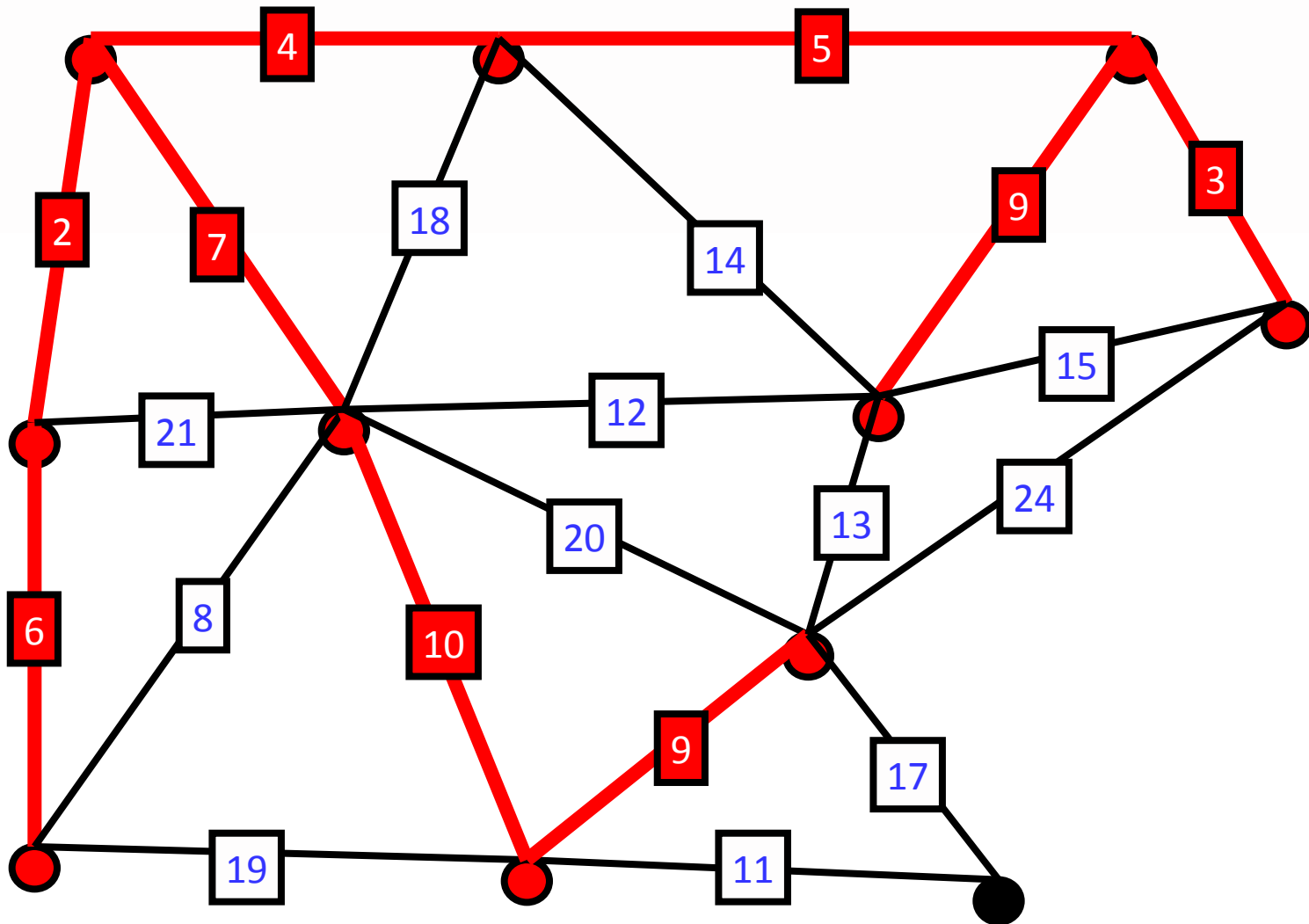
Sorted weights: 2,3,4,5,6,7,8,9,9,10,11,12,13,14,15,17,18,19,20,21,24



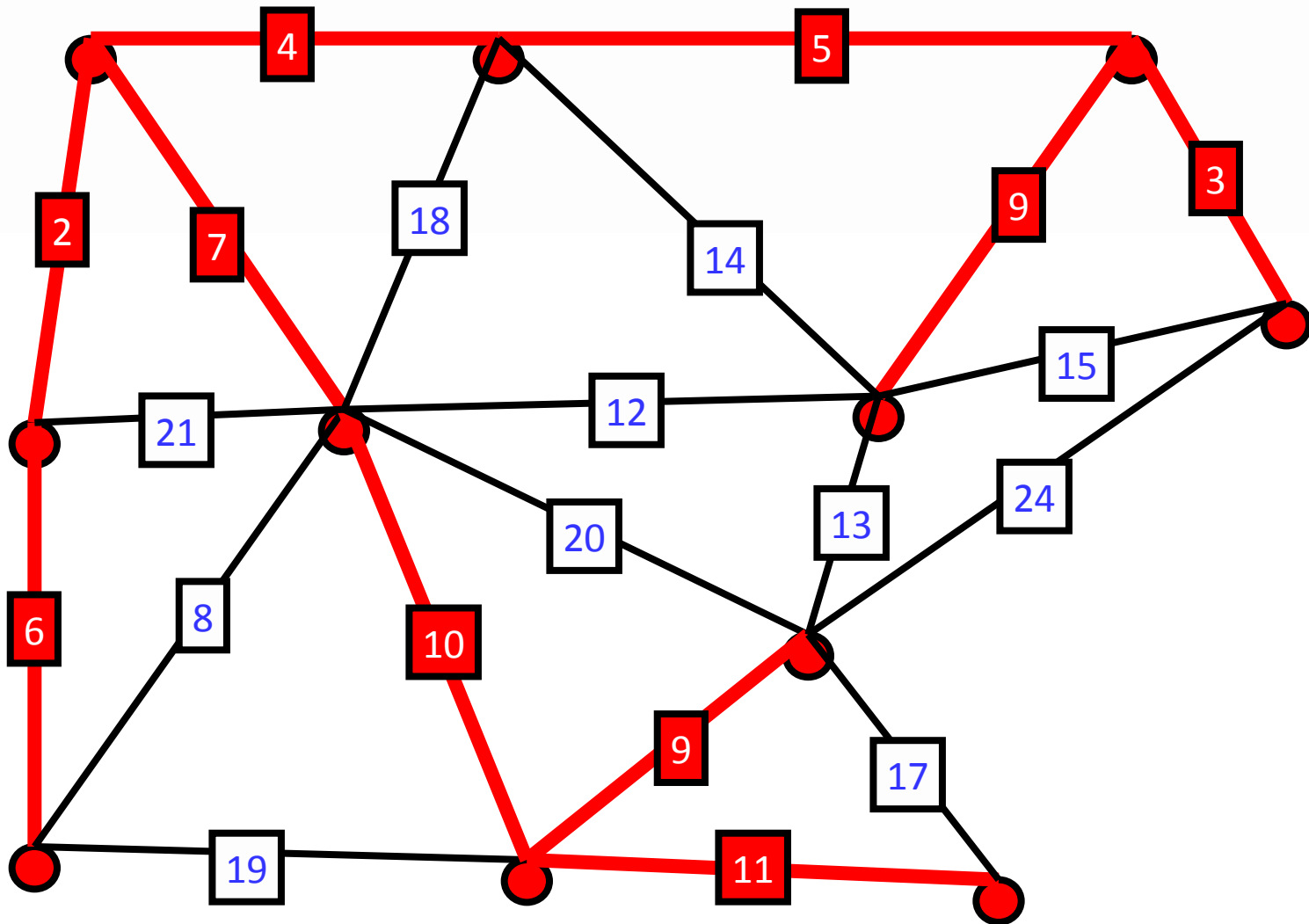
Sorted weights: 2,3,4,5,6,7,8,9,**9**,10,11,12,13,14,15,17,18,19,20,21,24



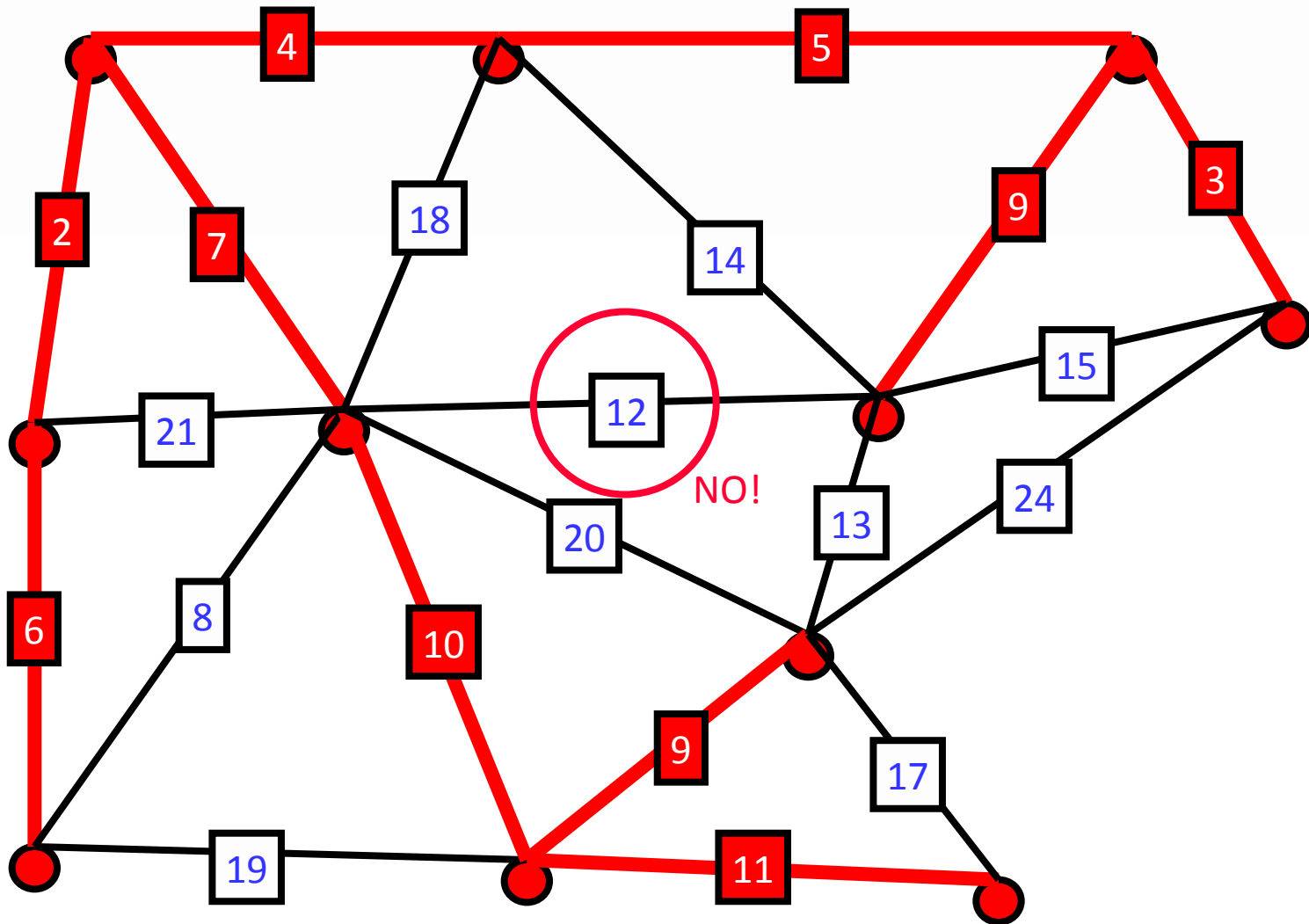
Sorted weights: 2,3,4,5,6,7,8,9,9,**10**,11,12,13,14,15,17,18,19,20,21,24



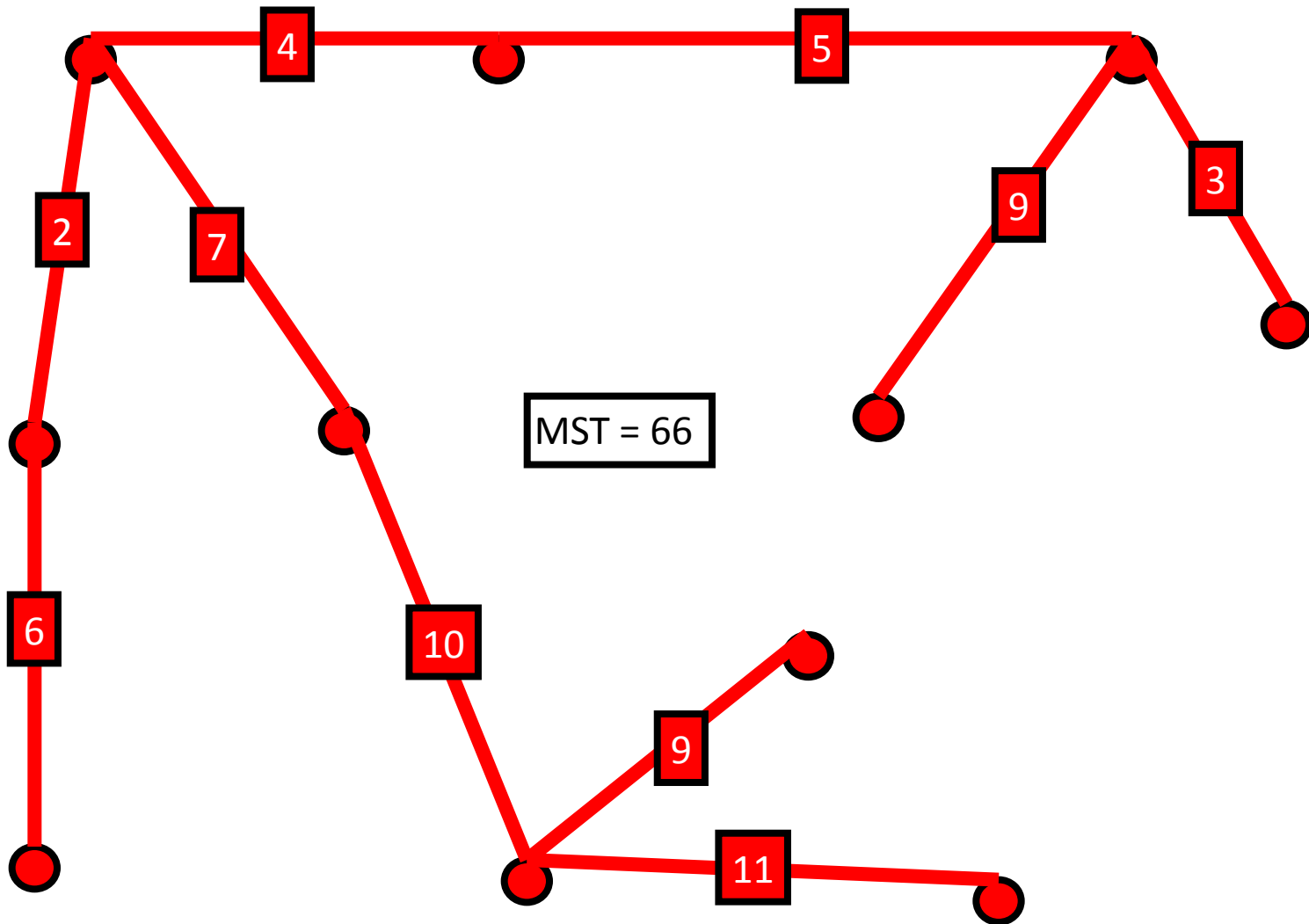
Sorted weights: 2,3,4,5,6,7,8,9,9,10,**11**,12,13,14,15,17,18,19,20,21,24



Sorted weights: 2,3,4,5,6,7,8,9,9,10,11,**12**,13,14,15,17,18,19,20,21,24



Kruskal's Algorithm



Kruskal's Algorithm: Correctness

Why does Kruskal's algorithm return a **spanning tree**?

1. Why is it **acyclic**?
2. Why is it **connected**?

Now we need to show it returns a **minimum** spanning tree.

Again, we'll use an **exchange** argument.

Kruskal's Algorithm: Correctness

Let **ALG** be the output of Kruskal's algorithm

Let **OPT** be an optimal solution (if **ALG**=**OPT**, we are done)

Consider the **first edge** e_{diff} added to **ALG** that's not in **OPT**.

$e_1, e_2, \dots, e_{\text{diff}}, \dots = \text{ALG}$ (ordered by weight)

$e_1, e_2, \dots, f_1, f_2, \dots = \text{OPT}$

same as ALG other edges (may or may not be in ALG, no particular order)

Key Claim: There's an optimal solution **OPT'** containing $e_1, e_2, \dots, e_{\text{diff}}$.
(will prove)

Consequence of Claim: We moved the “first difference” point later!
(I.e. first difference point of **ALG**→**OPT'** is later than that of **ALG**→**OPT**)

Keep repeating $\Rightarrow$ **ALG** is an optimal solution!

Kruskal's Algorithm: Correctness

Key Claim: There's an optimal solution OPT' containing $e_1, e_2, \dots, e_{diff}$.

Proof of Claim:

Goal: Take OPT and **exchange** out **some edge** f^* for e_{diff} to form another optimal solution OPT' . That is:

- OPT' should be a spanning tree
- $\text{weight}(OPT') \leq \text{weight}(OPT)$
(actually must be = since OPT is optimal)

Which edge f^* should we exchange out of OPT ?

Proof of Claim (continued):

Why is OPT' connected?

Why is OPT' acyclic?

Why is $\text{weight}(\text{OPT}') \leq \text{weight}(\text{OPT})$?

Want to show: $\text{weight}(\mathbf{e}_{\text{diff}}) \leq \text{weight}(\mathbf{f}^*)$.

Other independent inventors of MST algorithms:

Jarník, Prim, Borůvka



First used for designing electrical networks in 1926!

Today, a number of algorithms for clustering data (useful in machine learning) are variations of Kruskal's algorithm.

Running time (we won't prove) of Kruskal's algorithm: $O(m \log n)$
- need to analyze a data structure for detecting cycles

Best-known running time (Chazelle 2000): $O(m \cdot \alpha(n))$

Open problem: $O(m)$?

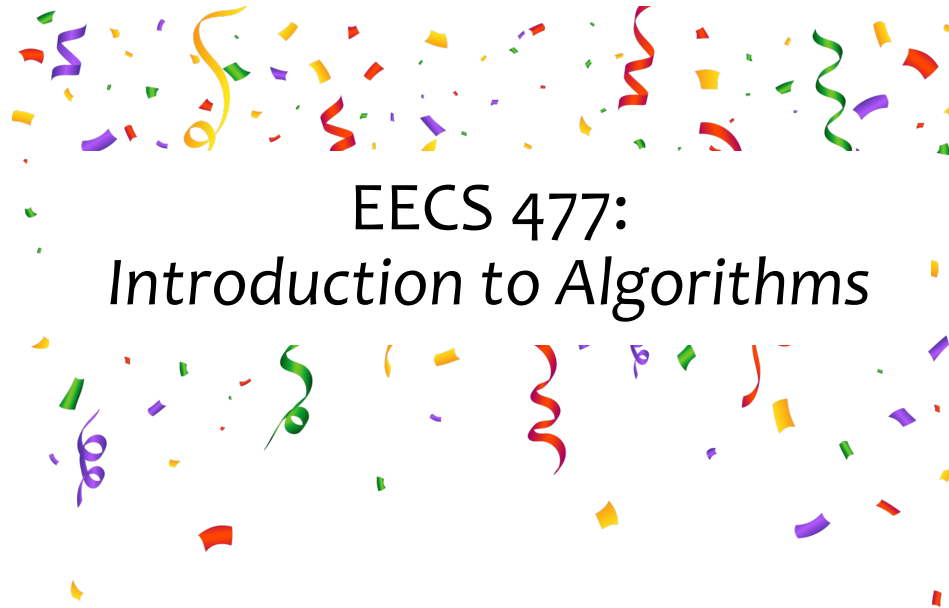
Inverse Ackermann
function: an extremely
slow growing function:
 $\alpha(n) \leq 4$ even when n is
particles in known
universe

Seth Pettie and Vijaya Ramachandran (2002) gave an asymptotically
fastest possible algorithm. But nobody knows how fast it is!



You've just finished a crash course on algorithms!

If you found the material interesting, try:



Next up: **Computability**

- What is a computer?
- What problems can a computer solve?
- What problems can a computer **not** solve?